

ft_transcendence — Kop

Récapitulatif des points de modules · sujet **v21.2**

Établi le 17 août 2026 par relecture du code de la branche `main`, exigence par exigence.

Objectif requis	14 points
Points validés (modules fonctionnels et démontrables)	16 points
Marge par rapport à l'objectif	+2 points
Points additionnels si les candidats sont complétés	jusqu'à +7 (total possible 23)
Barème	Majeur = 2 pts · Mineur = 1 pt

Règle du chapitre IV

« During evaluation: You will be asked to demonstrate each claimed module. Only fully functional and properly implemented modules will be counted toward your final score. Non-functional or incomplete modules = 0 points. »
— Seuls les modules réellement démontrés le jour de la soutenance comptent.

Partie obligatoire (chapitres III.2 et III.3)

Le non-respect d'un seul de ces points entraîne le rejet du projet. Chacun a été vérifié sur le code et sur l'application en fonctionnement.

Exigence du sujet	État	Vérification
Application web : frontend, backend, base de données	OK	Next.js 16 · Django 5.2 + DRF · PostgreSQL 16
Commits de tous les membres, messages clairs, répartition du travail	OK	<code>git shortlog</code> : les 4 membres présents, conventional commits
Conteneurisation, démarrage en une seule commande	OK	<code>docker compose up --build</code>
Compatible avec la dernière version stable de Chrome	OK	suite E2E exécutée sur Chrome 151
Aucun warning ni erreur JavaScript dans la console	OK	11 pages vérifiées automatiquement (<code>e2e/test_10_console.py</code>)
Pages Privacy Policy et Terms of Service, contenu réel, accessibles	OK	238 et 224 lignes, liens dans la sidebar de toute l'app et sur login/register
Multi-utilisateur : actions concurrentes, aucune corruption ni race condition	OK	3 tests à threads réels (<code>api/test_concurrency.py</code>) sur bonus, roue et paris
Mises à jour temps réel répercutées sur tous les clients connectés	OK	Django Channels : chat de ligue, messages privés, notifications
Frontend clair, responsive, utilisable sur tous les appareils	OK	3 résolutions × 6 pages, aucun débordement horizontal (<code>e2e/test_11_responsive.py</code>)
Framework ou solution CSS	OK	Tailwind CSS 4, tokens centralisés dans <code>globals.css</code>
Secrets dans un <code>.env</code> ignoré par Git + <code>.env.example</code> fourni	OK	seul <code>.env.example</code> est versionné

Schéma de base clair, relations bien définies	OK	17 tables documentées dans le README, 10 apps Django
Inscription et connexion sécurisées, mots de passe hachés et salés	OK	PBKDF2 (défaut Django) + 4 validateurs de mot de passe
Validation de tous les formulaires côté frontend et backend	OK	erreurs affichées côté client, 24 points de validation dans les serializers
HTTPS pour toute connexion au backend	OK	Caddy en terminaison TLS ; ports 3000 et 8000 non publiés

README (chapitre VI)

Exigence	État	Emplacement
Première ligne en italique avec les logins 42	OK	ligne 1
Description, Instructions, Ressources (dont usage de l'IA)	OK	sections dédiées
Team Information avec rôles PO / PM / Tech Lead	OK	tableau des 4 membres
Project Management : organisation, outils, communication	OK	GitHub + Discord, sessions communes
Technical Stack avec justification des choix	OK	4 justifications détaillées
Database Schema, Features List, Modules, Individual Contributions	OK	sections dédiées
Rédigé en anglais	OK	intégralement

Modules validés — 16 points

Catégorie	Module	Type	Pts	Exigences du sujet et réalisation
Web	Framework front + back	Majeur	2	Framework frontend et backend. Next.js 16 (App Router, rendu serveur, routes serveur) et Django 5.2 + DRF.
Web	Temps réel (WebSockets)	Majeur	2	Les trois exigences sont couvertes : <i>mises à jour temps réel</i> (3 consumers — chat de ligue, messages privés, notifications) ; <i>gestion propre des connexions/déconnexions</i> (refus si non authentifié ou non membre, <code>group_discard</code> à la déconnexion, reconnexion automatique côté client) ; <i>diffusion efficace</i> (groupes de la couche Channels sur Redis).
Web	Interaction entre utilisateurs	Majeur	2	Les trois minimums exigés : <i>chat</i> (privé + ligue, historique persisté), <i>profils</i> (page publique consultable), <i>amis</i> (ajout, suppression, liste).
Web	ORM	Mineur	1	Django ORM sur les 10 apps, migrations versionnées, aucune requête SQL brute.
Web	Système de notifications complet	Mineur	1	Exigence : <i>toutes</i> les actions de création, mise à jour et suppression. Création : demande d'ami, invitation de ligue, message privé. Mise à jour : règlement d'un pari. Suppression : suppression d'une ligue. Push WebSocket + suivi de lecture.
Web	Design system sur mesure	Mineur	1	Minimum exigé : 10 composants réutilisables, palette, typographie, icônes. 26 composants , tokens de couleur et de typo dans <code>globals.css</code> , jeu d'icônes maison.
User Mgmt	Gestion utilisateur standard	Majeur	2	Les quatre exigences : <i>modification du profil</i> ; <i>upload d'avatar avec avatar par défaut</i> ; <i>ajout d'amis et statut en ligne</i> (déduit de l'activité récente, fenêtre de 5 min) ; <i>page de profil</i> .
User Mgmt	Système d'organisation	Majeur	2	Liges privées. <i>Créer, modifier, supprimer</i> une organisation ; <i>ajouter</i> des membres (invitation + acceptation) ; <i>retirer</i> des membres (exclusion, départ volontaire) ; <i>consulter</i> avec classement interne. Modification et suppression réservées au créateur.
User Mgmt	OAuth 2.0	Mineur	1	Google via allauth + dj-rest-auth, tokens déposés en cookies httpOnly.
User Mgmt	2FA complète	Mineur	1	Second facteur obligatoire à chaque connexion : code à 6 chiffres tiré avec <code>secrets</code> , TTL de 5 minutes, envoyé par Brevo, session ouverte seulement après validation.
Gaming / UX	Gamification	Mineur	1	Minimum exigé : 3 mécaniques parmi 6. 4 présentes : badges, classements, défis quotidiens, récompenses. Persistées en PostgreSQL, barres de progression et retour visuel, règles de progression explicites.
Total validé			16	pour 14 requis

Candidats — jusqu'à +7 points

Modules partiellement présents. Ce qui manque est indiqué précisément : sans ces éléments, la règle « incomplet = 0 point » s'applique.

Module	Type	Pts	État	Ce qu'il reste à faire
Modules of choice — moteur de cotes	Majeur	2	Fonctionnel	Le code est complet et fonctionne. Le sujet impose une justification écrite dans le README couvrant quatre points : pourquoi ce module, quels défis techniques, quelle valeur ajoutée, pourquoi le statut majeur. Ce paragraphe n'existe pas encore — c'est le seul obstacle, et le meilleur rapport effort/points restant.
Web — SSR	Mineur	1	Fonctionnel	Next.js App Router rend déjà chaque page côté serveur. À présenter comme un choix explicite et documenté, pas comme un effet de bord du framework.
Web — Recherche avancée	Mineur	1	Partiel	Recherche par équipe/compétition et filtres par championnat présents ; affichage paginé progressivement côté client. Manque un tri exposé à l'utilisateur.
Web — Upload de fichiers	Mineur	1	Partiel	Avatar avec double validation (type et taille, client et serveur) et stockage contrôlé. Manquent : plusieurs types de fichiers, prévisualisation, indicateur de progression, suppression du fichier.
User Mgmt — Permissions avancées	Majeur	2	Partiel	Trois rôles (owner , admin , user), panneau d'administration, vues et actions différenciées selon le rôle. Manque le D de CRUD : aucune route ne supprime un utilisateur (seule la suppression d'une relation d'amitié existe).

Modules non commencés — 0 point

API publique avec clé et rate limiting · Accessibilité WCAG 2.1 AA · Internationalisation (i18n) · Support RTL · Navigateurs supplémentaires · PWA · Fonctionnalités collaboratives temps réel · Modules d'intelligence artificielle · Cybersécurité (WAF + Vault) · Jeux et modules dépendants d'un jeu · DevOps (ELK, Prometheus, microservices) · Dashboard analytique · Export/import de données · Conformité RGPD · Blockchain.

Statistiques de jeu et historique de matchs

Ce module mineur exige explicitement qu'un module « jeu » ait été implémenté. Kop n'est pas un jeu au sens du sujet : il ne peut donc pas être revendiqué, quelle que soit la richesse des classements et de l'historique de paris.

Vérifications automatisées

Suite	Résultat	Portée
<code>docker compose exec backend python3 -m pytest</code>	252 passed	modèles, vues, permissions, concurrence
<code>./e2e/run.sh</code>	28 passed	parcours utilisateur complet, console, responsive
<code>docker compose exec frontend npx tsc --noEmit</code>	0 erreur	typage strict
<code>manage.py makemigrations --check</code>	à jour	aucune migration en attente
<code>manage.py spectacular</code>	0 erreur	schéma OpenAPI

Sur les tests de concurrence

L'exigence « no data corruption or race conditions » est vérifiée par des tests à threads réels, et non par simple lecture du code : quatre requêtes simultanées sont lancées sur une barrière commune. Le verrou `select_for_update()` a été retiré temporairement pour contrôler que les tests échouent bien sans lui — sans verrou, les quatre requêtes de bonus quotidien aboutissent toutes, soit un quadruple crédit.

Document généré à partir du code de la branche `main`, chaque exigence confrontée au sujet v21.2. Les points marqués « OK » ont été vérifiés dans le code ou par un test automatisé, pas par déclaration.